

Delphi App Translation Studio

Engineering Guide

How the product is built, why it is built that way,
and what holds it together

Last changed: August 21, 2026

Applies to: alpha, build 2026.08.18.128

Frameworks: Delphi VCL and FireMonkey · Platform: Windows (Studio)

Written from the source. Every number, file name, and unit reference in this document was taken from the code as it stands on the date above.

Contents

Headings use the built-in Heading 1–3 styles, so a live page-numbered table of contents can be inserted over this list at any time through *References > Table of Contents*.

1. Purpose and audience
2. What the product does
3. Design principles
4. Architecture
5. Core types and the workspace
6. The scan
7. Context: telling the service what a string means
8. The catalog and its validator
9. Translation providers
10. Terminology, dictionaries, and hyphenation
11. The layout planner
12. Right-to-left layout
13. The runtime pack
14. The runtime
15. Contracts: how this product is kept honest
16. Extending the product
17. Build and release validation
18. Known limits

Appendix A — Unit inventory

Appendix B — Contract inventory and framework parity

Appendix C — Files, folders, and where things live

1. Purpose and audience

This guide is for the engineer who has to change this product: to fix a defect, add a capability, or judge whether a proposed change is safe. It assumes fluency in Delphi and no prior knowledge of this codebase.

It is not a user guide. It does not explain how to run the Setup Wizard; it explains what the Wizard is doing and why.

It is organized to be read straight through once and then used as a reference. Sections 4 through 14 follow the path a string takes through the product, in order: scanned out of a form, described, translated, measured, planned, exported, and applied. Sections 15 and 16 describe how the work is verified and how to extend it.

Where a design decision was reached by measurement rather than by reasoning, the measurement is given. Where something is known to be incomplete, it is said so plainly and pointed at the Fix List rather than glossed over.

2. What the product does

Delphi App Translation Studio takes a Delphi application and produces the files needed to run that application in another language, including the layout adjustments the new language requires, because translated text is rarely the same size as the original.

The product performs six jobs in sequence:

- reads the application's forms and source and finds the text a user will see;
- works out what each string *means* in that application, without anyone typing a description by hand;
- translates through DeepL or Google Cloud Translation using the developer's own API key;
- measures the translated text with the same engine the framework will draw it with, then plans the width, height, wrapping, font size, and position changes it needs;
- writes an offline JSON language pack;
- and applies that pack at run time through a component, so a user can change language while the application is running.

The deployed application needs no API key and no internet connection. Everything that requires a network happens once, on the developer's machine.

3. Design principles

Almost every design decision in this codebase follows from one of five commitments. When a change seems to conflict with the surrounding code, it is usually because it conflicts with one of these.

3.1 The target application is evidence, never a workpiece

The product reads the target project's `.dfm`, `.fmx`, `.pas`, `.dpr`, and `.dproj` files and never writes to them. Every artifact it produces lives outside the target tree. This is not a configuration option; it is the reason a developer can point the tool at a working application without taking a risk.

It has a consequence that recurs throughout this guide. Where a translation genuinely requires a change to the application, such as a `TranslateText` call for a string the application composes itself, the product's job is to *say so precisely*, not to make the change.

3.2 The runtime is offline

Translation happens once, at development time. What ships is a JSON pack read from disk. No key, no network, no telemetry. This separates the one phase that needs the internet from the one that must never depend on it, and it means a translated application behaves identically on a machine with no network at all.

3.3 Artifacts are readable and diffable

The catalog, the layout proposal, and the runtime pack are JSON. They can be read in a text editor, compared between versions, committed to source control, and corrected by hand. Several defects described in this guide were diagnosed by reading those files rather than by debugging, which is an argument for the format on its own.

3.4 One decision lives in one place

The list of layout properties a pack may carry once existed in four units. They drifted, and each drift silently deleted a feature: the planner decided correctly, the runtime would have applied correctly, and the pack in between simply did not carry the value. There was no error anywhere, and nothing failed.

That list now lives only in `DAT.Runtime.LanguagePack`, which the exporter and both applicators already reference. Section 15 describes the tests that guard the joins where copies like that tend to reappear.

3.5 Both frameworks, one implementation

VCL and FireMonkey differ in defaults, in property names, and in what the framework does on your behalf. The response is not two code paths but one implementation behind a seam. The clearest example

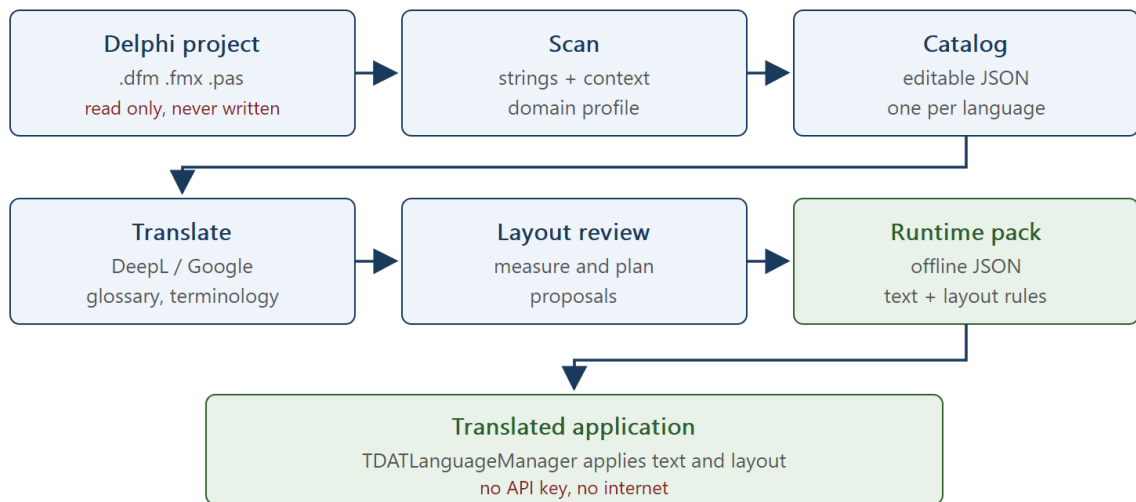
is text measurement (section 11.1). The clearest payoff is right-to-left mirroring (section 12), where the VCL has native support, FireMonkey has none at all, and both are served by the same planner pass.

4. Architecture

4.1 The pipeline

Figure 1 — The translation pipeline

Each stage writes a durable artifact. Nothing downstream reads the target application.



Each stage writes a durable artifact and the next stage reads it. That is what makes the pipeline diagnosable: when something is wrong on screen, the question is always *which artifact first contains the mistake*, and every one of them can be opened and read.

The stages are deliberately not fused. It is possible to scan without translating, to translate without planning layout, to re-plan layout without re-translating, and to rebuild a pack from an edited catalog. Each of those is a real workflow, and each is possible only because the intermediate artifacts are real files.

4.2 Source layout

Folder	Units	Lines	Responsibility
source\core	10	3,497	Catalog types, JSON, workspace paths, glossary, hyphenation, pack builder
source\scan	10	3,741	Reading forms and Pascal; context and domain profiling
source\review	5	4,720	The layout planner, the text measurement seam, code-owned geometry
source\provider	8	1,371	Machine translation: placeholders, batching, language codes, retry, credentials

source\runtime	5	4,452	What ships inside the customer's application
source\components	7	2,064	The language manager component and the language selectors
source\studio	4	5,204	The Studio and the Setup Wizard
source\integration	5	2,172	Component kits, packages, deployment
source\validation	1	403	The catalog validator

Fifty-seven units, about 27,700 lines.

4.3 The runtime boundary

The division that matters most is **runtime versus everything else**. Units under `source\runtime` and `source\components` are compiled into the customer's application. Everything else runs only in the Studio and never ships.

The runtime units are held to stricter rules than the rest of the codebase. They must be small, must not reach for the network, must not depend on any Studio unit, and must never write to a user's disk uninvited. A dependency accidentally added from a runtime unit to a Studio unit would pull the entire translation pipeline into every customer application, so the direction of that dependency is worth checking whenever a runtime unit gains a new `uses` clause entry.

5. Core types and the workspace

5.1 The catalog data model

`DAT.Core.Types` holds the types every other layer passes around. The central one is the translation entry, and knowing its fields explains most of what the rest of the product does with it.

Field	Holds	Used by
key	Form, component, and property path, such as <code>frmMain.btnSave.Caption</code>	The runtime, to find the control again
sourceText	The original text	Translation; returning to the source language
translatedText	The translation	Everything downstream
componentClassName	The control's class	Context; the planner's per-class rules
sourceFileName, sourceLine	Where the string was found	Review; reporting strings the application must handle itself
textOwnership	Who controls the string at run time (section 6.5)	Deciding whether the pack can apply it at all
contextKind and context sentence	What the string means in this application	The translation request
sourceChecksum	A fingerprint of the source text	Detecting that a form changed under a saved decision
status	Untranslated, machine translated, reviewed, approved	Review; validation

The catalog itself adds the application identity, the framework, the source language, and the target locale, including the date, time, and number formats that language expects. The locale block matters more than it looks: applying a language is not only a matter of words, and the runtime sets format settings from it.

5.2 The workspace

`DAT.Core.TranslationWorkspace` owns every path the product writes to. No other unit composes a path by hand, which is what makes it possible to state in one place that nothing is ever written inside the

target project.

Each project gets a workspace folder holding a development catalog per language, the built packs, and the saved layout decisions. Appendix C lists the locations.

5.3 Project detection

`DAT.Core.ProjectDetection` reads a `.dproj` to establish two things: which units and forms belong to the project, and which framework it targets. The framework answer decides which text measurement engine the planner will use, which property names appear in the pack, and which applicator the customer's application will link, so it is the first fact everything else depends on.

6. The scan

`DAT.Scan.Project` drives the scan. `DAT.Scan.FormText` reads designer files, `DAT.Scan.PascalResources` reads Pascal, and `DAT.Scan.Rules` classifies what they find.

6.1 What is collected, and what is deliberately not

The scan is limited to project-referenced source units and designer resources. Broad harvesting of `Items.Add`, `Lines.Add`, `TextOut`, and similar calls is **disabled on purpose**: in practice those carry data rows, file names, log lines, and generated content rather than stable interface text. An earlier version harvested them and produced thousands of strings that no user would ever see, which buries the few hundred that matter.

Designer-authored `Items` and `Lines` are still collected, because those come from the form file, where a developer put them deliberately.

6.2 Reading designer files

One parser reads both `.dfm` and `.fmx`. They are the same format with different property names, so the parser tracks object nesting and property assignment generically, and the framework differences are handled by name mapping: `Caption` against `Text`, `Left` against `Position.X`, `Width` against `Size.Width`, and so on.

Fonts are the awkward case. A `.dfm` records a font as `Font.Height` in negative pixels rather than `Font.Size` in points, and the designer writes `Font.Size` only if somebody typed it. Reading only `Font.Size` therefore leaves every VCL control at the default, and the planner sizes boxes for text nobody will see. The conversion is `Round(-Height * 72 / 96)`, rounded to match what `TFont` itself does, and fonts inherit down the object tree from the form.

6.3 Collections and nesting

Grid columns are worth understanding before changing the parser. A VCL grid keeps its heading on the column's `Title`, so the catalog key reaches one level further in than the column itself:

```
grdSchedule.Columns[0].Title.Caption
```

Collections nest inside objects, and a collection item's `end` must not pop the object stack. Getting that wrong makes every control after the grid appear to be a form, which corrupts the rest of the file silently. A contract exists purely to hold that behavior in place (Appendix B).

FireMonkey has no equivalent hazard, because a FireMonkey column is an ordinary named component with a `Header` property rather than a collection item. The same capability is reached by a completely different path on each framework, which is a pattern worth expecting throughout the runtime.

6.4 Reading Pascal

The Pascal reader collects resource strings and string constants assigned to recognized interface properties. It records the file and line for everything it finds, which is what later allows the product to tell a developer exactly where a string it cannot handle actually lives.

6.5 Text ownership

Every string is classified by who controls it at run time. The classification decides what happens to the string for the rest of the pipeline.

Classification	Meaning	Applied automatically?
designerAutomatic	Set in the form file and not touched by code	Yes
runtimeWired	Assigned in code, but through a path the runtime can reach	Yes
runtimeUnwired	Assigned in code, composed at run time	No — needs a call in the application
applicationData	A data value, not interface text	No
suspicious	Looks like data, or like something that should not be translated	No

The third row is the one that surprises people. Where an application builds a caption in code and reassigns it whenever the display refreshes, anything the pack writes there is overwritten moments later:

```
StatusBar1.Panels[1].Text := 'Items in list: ' + ItemCount.ToString;
```

The classification is correct, and the translation is genuinely impossible without a `TranslateText` call in the application, which principle 3.1 forbids the product from adding. What the product does *not* yet do is report those strings to the developer, even though it holds the file and line for each one. That gap is on the Fix List.

6.6 Encoding

`DAT.Scan.TextCodec` handles the fact that a Delphi source file without a byte order mark is read in the machine's ANSI codepage. This is not a theoretical concern: non-ASCII characters in a source file that lacks a BOM arrive wrong, and the failure is silent. Section 15.5 describes the build guard that enforces the rule across this product's own source.

7. Context: telling the service what a string means

A machine translation service sees one string at a time. A short interface string carries almost no information on its own, and the service has to guess: a word that names a thing in one program is a verb in another, and a term that means one thing in a media player means something else entirely in a disk utility.

The product's answer is that everything needed to settle those questions is already in the scan. Nobody has to type it.

7.1 The application domain profile

DAT.Scan.DomainProfile reads the application rather than trying to recognize it. An earlier version matched a handful of subjects from keyword lists. It read some applications correctly and would have failed file utilities, database tools, point of sale, laboratory systems, and most of the long tail Delphi is actually used for. A recognizer can only recognize what somebody already thought of.

Two things come out of reading an application:

- **Its own vocabulary**, with the furniture every program shares (File, Edit, Cancel, Save, OK, Close) set aside. What remains is characteristic: *invoice, ledger, posting, reconcile* for an accounting package; *rename, mask, extension, folder* for a file utility.
- **Which sense of an ambiguous word this application means.** A shared list names the English words that are ambiguous in a user interface and gives each sense the evidence words that would be present if it were the one meant. The application's own vocabulary casts the vote.

7.2 Word sense

Volume is loudness in an application that also says mute and speaker, and a disk in one that says partition and format. *Mask* is a filename pattern where the application talks about filenames. Nothing in the code has to know what kind of application it is looking at; the vocabulary decides.

Where the application settles nothing, **nothing is said**. A guess between two senses is worse than silence, because it reaches the service as a confident instruction to be wrong.

7.3 Part of speech

A button says what pressing it will do, so its caption is an instruction. A menu item names a thing. English hides the difference, because for most verbs the imperative and the dictionary form are the same word. Many other languages do not, and a service given a bare word with no indication of which is wanted may return a grammatically correct statement where an imperative was needed.

DAT.Scan.Context therefore states which is wanted, chosen from the control class that every string

already carries: an imperative for a button, the form the language uses on menus for a menu item, a noun phrase for a column heading or a label.

7.4 Delivering context to the string it describes

A service accepts one context per request. An implementation that batches fifty strings into one request therefore has one context field for fifty different descriptions, and concatenating them means every string arrives wearing forty-nine descriptions of other controls. The context is present, paid for, and diluted into uselessness.

The economics make the fix free. **Billing is per translated character, not per request**, and context characters are not billed at all. A mid-sized application costs the same number of billed characters whether it goes as six requests or as three hundred.

DAT.Provider.Batching therefore groups strings by *identical* context. Where a context is unique the group holds one string; where many strings share a context, or have none, they travel together as before. The only cost is round trips, paid once per language for a result that is then stored permanently.

8. The catalog and its validator

`DAT.Core.CatalogJson` reads and writes the catalog, which is the editable record of one application in one language. Each entry carries the fields listed in section 5.1. The catalog is meant to be edited: a reviewer can correct a translation in the Studio or in a text editor, and the correction survives a re-scan because entries are matched by key.

`DAT.Validation.Catalog` refuses to let a pack be built from a catalog with errors. This is not decoration. The most valuable class of error it catches is damage to format specifiers: a translation service may reorder, alter, or destroy a `%s` or `%.2f`, and a format string that reaches production damaged will fault or print nonsense at users indefinitely. The validator is frequently the only thing standing between a plausible-looking translation and a broken application.

Validation is a gate, not a warning. A catalog with blocking errors cannot be exported.

9. Translation providers

Two services are supported, both using the developer's own API key: **DeepL** and **Google Cloud Translation**. The key is held in Windows Credential Manager, or kept for the session only, and is never written to the catalog, the pack, or any log.

`DAT.Provider.Client` is the transport. Four units sit around it, and each one exists because of a distinct failure mode that a naive client walks straight into. Each is worth understanding before changing that code.

9.1 Placeholders

`DAT.Provider.Placeholders` lifts format specifiers out of a string before the request and replaces each with a token that carries its own index, so a token the engine moves still comes back identifiable as the specifier it stood for. Engines do move them; right-to-left target languages move them routinely.

Identical specifiers get separate tokens. A string containing three occurrences of `%.2f` cannot be restored by search and replace, because there is no way to tell which returned token was which.

Afterward the specifiers are restored and checked. If they do not match, the **source text is returned** rather than the translation. An untranslated string among translated ones is obvious in review; a damaged format string is invisible until a customer sees it.

A string that is *nothing but* specifiers is never sent at all. `%.2d/%.2d` is a date format, not a sentence, and there is nothing in it to translate.

9.2 Language codes

`DAT.Provider.LanguageCodes` converts a catalog's language code into one the service will accept. A catalog names languages the way Windows does, with a language and a region. Services accept a two-letter code plus a published list of regional variants, and reject anything else with a hard error.

A normalizer that passes the region through unchanged works for whichever languages happen to be on that list and can never work for the rest. The region is now kept only where the service is documented to accept it and dropped everywhere else. **Dropping is the safe direction:** the general code works for every language the service supports, so a language added after this was written still translates rather than failing.

9.3 Retry and pacing

`DAT.Provider.Retry` treats a rate-limit response as what it is: the service asking for a slower pace. The only wrong answer is to stop. Six attempts, waiting one second, two, four, eight, and sixteen — thirty-one seconds of patience. A `Retry-After` header is obeyed to the second where the service sends one, capped so that a large value cannot stall a run, and treated as absent rather than guessed at when it arrives in a form the client cannot parse.

Errors that will not improve with time are *not* retried. A malformed request, a refused key, and an exhausted quota all come back just as fast the second time.

The client also paces itself. The delay between requests starts at zero, grows by a quarter second each time a rate limit is met, and holds for the rest of the run. A run that is never refused pays nothing; a run that is refused once slows down instead of arguing with the service.

9.4 Reporting what the service said

Both services return an explanation in the response body, and an early version of the client discarded it in favor of a generic message listing six things that might be wrong. That is worse than useless: it tells a developer to check everything. Rejections now quote the service's own explanation and add one line where the status code carries meaning of its own.

10. Terminology, dictionaries, and hyphenation

10.1 The three shared stores

Three shared, editable stores live outside any one project, so that work done on one application benefits the next. All three are plain JSON and are meant to be corrected by hand.

Store	Scope	Purpose
Dictionaries\<lang>.json	Per language	Approved wording earned on one application, available to every later one
Terms\ambiguous-terms.json	One file, English	Words ambiguous in a user interface, with the evidence that settles each sense
Hyphenation\<lang>.json	Per language	Where a long word may be broken

A per-project glossary (DAT.Core.Glossary) sits above these for terms specific to one application, and DAT.Core.Terminology resolves the two against each other when a request is built.

10.2 Hyphenation

Some languages build a single long word where English uses three, and a single word cannot wrap: there is no space in it for a control to break at, so it is simply cut off at the edge of its box. No amount of widening fixes the general case, because the next word may be longer still.

DAT.Core.Hyphenation holds a per-language dictionary describing which letters are vowels, which consonant groups form a single sound, and how much of a word must be left whole at each end. From that it marks every point where the language allows a break.

The marks are carried in the pack as soft hyphens (U+00AD), because nothing at build time knows how wide a control will end up. They are applied to **captions only**. Format strings are left exactly as written, since their text goes on to be filled with data and a mark in the middle of a specifier would corrupt it.

10.3 How the two frameworks differ on soft hyphens

This is the clearest example in the product of two frameworks requiring opposite treatment, and it was measured rather than assumed.

	FireMonkey (DirectWrite)	VCL (GDI)
--	--------------------------	-----------

A mark that is not used	Invisible. A marked word measures exactly what the unmarked word measures.	Drawn as an ordinary hyphen. The word measures wider for every mark in it.
A mark that is needed	Used as a break opportunity. The word breaks at a syllable.	Ignored. DrawText will not break a line at U+00AD.
What the runtime does	Nothing. The marks reach the caption and the renderer chooses.	Resolves every mark before it reaches a caption.

So `DAT.Runtime.VCL` resolves the marks itself, and does it at the last possible moment: after every layout rule has been applied and each control is the size it will really be. A control that wraps gets a real hyphen and a real line break at the last mark that fits its width; one that cannot wrap gets the plain word with the marks removed. **No soft hyphen ever reaches a VCL caption.**

`DAT.Runtime.FMX` deliberately contains no hyphenation code at all. This is not an omission, and a test asserts it: the renderer knows the final width and the pack never can, so leaving the choice to the renderer is both simpler and better.

11. The layout planner

DAT.Review.Localization is the largest and most consequential unit in the product. It decides what has to change so that translated text fits, and it produces a list of proposals rather than applying anything itself.

11.1 The measurement seam

Text is measured with the engine that will actually draw it.

Framework	Unit	Measures through
VCL	DAT.Review.TextMeasurement.GDI	GetTextExtentPoint32
FireMonkey	DAT.Review.TextMeasurement.FMX	TTextLayout
Either	DAT.Review.TextMeasurement	Chooses from the catalog's framework

The seam is one function wide: a measurer answers a width for a run of text at a point size and weight, and nothing else. Everything the planner builds on top of that number is framework-neutral and shared. Measurers register themselves as they are linked in, so the planner never names a framework unit.

DPI is pinned to 96, because that is the basis a form was designed at.

The two engines do not agree, and the difference is large enough to matter: the same string at the same point size measures roughly a quarter narrower through TTextLayout than through GDI. That is precisely why the seam exists. Measuring with the wrong engine produces a plan that is confidently wrong in both directions at once, failing layouts that are fine and passing layouts that are not.

11.2 Framework facts encoded in the planner

Each of these produces wrong layouts if it is not known:

- TLabel.AutoSize defaults to **True** on the VCL and WordWrap to False, which is the opposite of a FireMonkey label. A translated caption therefore stretches a VCL label before any rule is applied, so AutoSize must be cleared *before* the text is assigned, not after.
- Default font sizes differ: nine points for the VCL, twelve for FireMonkey. A fixture or a form that states no font is not the same size on the two frameworks.
- The VCL names a second instance of a form Name_1. Looking up a control by its instance name rather than by the form's identity leaves every duplicate dialog untranslated.
- TAlignLayout constants are not the VCL's TAlign constants, and the two enumerations do not share ordinal values. Alignment is mirrored by name, never by number.

11.3 The phases

The planner runs a sequence of passes over a single resolved model, so that the values it finally exports agree with one another rather than describing conflicting placements.

1. **Phase 1** — start every control from its designer geometry.
2. **Phase 2** — size each control against its measured translated text. Rows of buttons, captions above fields, and evenly pitched rows are treated as sets rather than as individuals, because a row where one member grew and the others did not reads as a mistake.
3. **Phase 3** — resolve collisions repeatedly against the planned geometry, and hold captions inside the frames drawn around them.
4. **Phase 3d** — the settling pass: level rows, widen and re-center headings, widen lone buttons rightward only, give stacked paragraphs one shared font size, and narrow wrapped text to the width its wrap actually uses.
5. **Phase 3e** — mirror the form for a right-to-left language (section 12).
6. **Phase 4** — emit proposals from the settled geometry.

Two of those deserve their reasoning stated. A button is widened rightward only because a button is positioned against the thing it acts on, so it keeps the place it was drawn in. Wrapped text is narrowed rather than left at its designed width because a box wider than its wrap uses puts nearly all the words on the first line and one or two on the second, which is the ragged result a typesetter spends a career removing.

11.4 Trial and revert

Every step in the settling pass is applied speculatively and undone if it breaks something. A change that would push a control off its form, over its neighbor, or outside its container is reverted rather than kept. This is what allows the settling rules to be written independently of one another: a rule does not have to know what the other rules want, because a rule that fights another one loses and the geometry returns to what it was.

11.5 Geometry the application owns

`DAT.Review.CodeGeometry` reads the Pascal unit beside each form and notes any control whose `Left`, `Top`, `Width`, `Height`, `Position.X`, `Position.Y`, `Align`, `BoundsRect`, or `SetBounds` is assigned there. Those controls have their text translated and their **geometry left entirely alone**.

The reason is that an application which positions a control in code has already decided where it goes, and usually decides once, at startup. The planner reads the designer geometry, which is not the geometry the application will actually use, and proposes a position that overwrites a decision the application will never make again. Returning to the source language then restores the *designed* position rather than the computed one, so the control ends up somewhere it has never been.

The detection is deliberately literal: an assignment to a named identifier's geometry property, and nothing cleverer. No expression analysis, no following of variables, no with statements. A control it misses behaves as it did before; a control it claims wrongly loses only its layout adjustments and is still translated. **Both directions fail softly**, which is the right property for a heuristic that reads somebody else's source.

One reading serves both frameworks, because what is being read is Pascal rather than VCL or FireMonkey.

12. Right-to-left layout

A right-to-left interface is a **reflected** interface, not reversed text. Producing correctly translated words in a left-to-right arrangement is worse than refusing the language outright, because it looks as though it worked.

12.1 Choosing the VCL reading mode

The VCL offers three right-to-left modes, and they are not interchangeable.

BiDiMode	Flips alignment	RTL reading	Left scroll bar
bdRightToLeft	yes	yes	yes
bdRightToLeftNoAlign	no	yes	yes
bdRightToLeftReadingOnly	no	yes	no

Under `bdRightToLeft` the framework flips text alignment on its own. Since the planner already decides alignment for every control, that flip lands on top of the planner's decision and silently undoes it, producing a double flip that looks like a defect in the planner. **`bdRightToLeftNoAlign` is therefore what the runtime uses:** reading order and scroll bar side from the framework, alignment from the planner.

Digits need no help. A right-to-left run followed by a number renders with the letters reversed and the number intact, in both renderers, because the Unicode bidirectional algorithm handles it. Version strings, times, quantities, and paths are safe.

12.2 Mirroring in the planner, not in the framework

The VCL has `BiDiMode` and `FlipChildren`. FireMonkey has neither. Leaning on the VCL's mechanism would mean writing the FireMonkey half separately and getting different behavior on each, which principle 3.5 rules out. The mirror is therefore computed by the planner and emitted as the ordinary position rules the runtime already applies.

The transform is parent-relative, which handles nesting without recursion:

```
MirroredLeft := ParentInnerWidth - (PlannedLeft + PlannedWidth)
```

A form is never mirrored. A window has no parent to be reflected within, so the arithmetic degenerates into negating the window's own screen position, which moves the window off the side of the display. The planner explicitly skips the record whose component name is its own form name.

12.3 What mirrors, and what deliberately does not

Mirrors	Does not
Coordinates, within each parent Align and TAlignLayout, for framework-placed controls Anchors, where exactly one horizontal edge is anchored Text alignment (center stays center) Grid column order Tab order Reading order and scroll bar side (VCL)	Transport buttons — rewind, play, and stop refer to the direction a recording moves, not the direction a language is read, so the group moves to the mirrored side as a block and keeps its internal order Numbers, times, versions, and paths — handled by the renderers Images — the transform only ever moves a control, never its contents, so artwork and logos are safe by construction

A control anchored to *both* horizontal edges stretches, which is already symmetrical, and is left alone. A control anchored to neither has nothing horizontal to change. Only the one-edge case is mirrored, and getting this wrong is invisible until the user resizes the window.

12.4 Order of application

Reading order is applied **before** the text, and the reason is subtle enough to be worth documenting. Translating a menu item's caption causes the menu to be rebuilt, and Delphi stamps each item with the reading order in force at the moment of that rebuild. Applying direction after the text therefore rebuilds the menu in the direction being left behind, and then depends on the framework's own notification to correct it.

That notification is not dependable here. `TMenu.DoBiDiModeChanged` begins:

```
if (not SysLocale.MiddleEast) or (WindowHandle = 0) then Exit;
```

Both conditions bite. The window handle is momentarily zero while the form's window is recreated, which is exactly when `BiDiMode` changes. And on a machine whose Windows locale is not configured for those languages, the VCL does not lay menus out right-to-left at all, whatever `BiDiMode` says, so menu direction can behave differently on two machines running the same build. Both are worth knowing before trying to reproduce a menu problem.

Setting direction first removes the dependency entirely: whatever is rebuilt afterward is rebuilt the right way round to begin with.

13. The runtime pack

`DAT.Core.RuntimePack` writes the pack, which is the only artifact that ships. Schema 3 carries the language and locale, the translated strings by key, the source text for each key so that returning to the source language is possible, runtime templates for strings the application formats itself, font colors, and the layout rules.

13.1 What may be exported

Two rules govern what may go into a pack:

- A proposal must be **accepted**. A rejection made in review is honored; a proposal left pending is not exported.
- A property must appear in the one allowed-property list, `IsRuntimeLayoutProperty` in `DAT.Runtime.LanguagePack`.

Both rules once had private copies elsewhere, and both silently deleted features. The pending case is the more instructive of the two, because it could not be fixed by fixing the code. `RestoreDecisions` reads the previous run's proposal file and copies each saved decision over the analyzer's, so that a rejection survives a re-scan. It also copied *pending* — and pending is not a decision, it is the absence of one. A proposal file written by a build that did not yet know a property existed recorded that property as pending, and every later run restored that over a freshly accepted decision. The feature was vetoed permanently by a stale file, and no amount of rebuilding the analyzer would have changed it. A saved *pending* now means "nobody has decided," and the analyzer's own judgment stands.

13.2 Why the source text ships too

The pack carries the source text for every key, not only the translation. Without it, selecting the source language again leaves the words in the last language chosen, however correctly the geometry is restored. This is easy to leave out and produces a fault that only appears on the second language change.

14. The runtime

`DAT.Runtime.VCL` and `DAT.Runtime.FMX` apply a pack to a live form. `DAT.Components.Core` holds the shared language manager behavior, and the two framework adapters differ only where the frameworks do.

14.1 Order of operations in `ApplyToForm`

1. Snapshot the original geometry, once, before any language is applied.
2. **Restore** from that snapshot.

3. Apply reading order (section 12.4).
4. Apply text, including collections such as grid headings.
5. Apply layout rules in a fixed order: `AutoSize`, font size, `WordWrap`, text alignment, `Align`, size, position, `Anchors`, tab order, column widths, column order.
6. Resolve soft hyphens, now that every control is its final size (VCL only).

The ordering is not arbitrary, and two steps in it are load-bearing. An auto-sizing label recomputes its own bounds from its text and discards an assigned width, so `AutoSize` must be cleared before anything else is set. Column widths name a column by its designed index, so the column *order* must be reversed last, or every width lands on the wrong column.

14.2 Snapshot and restore

Step 2 is the one that is easy to omit and hard to diagnose without. Applying a language must start from the form as it was designed, not from the form as the previous language left it. Otherwise each language inherits whatever the last one changed, and a rule that is simply *absent* from the new pack has nothing to undo it. The symptom is a form that drifts a little further wrong with every language change and is correct again after a restart.

The snapshot covers position, size, font size, `AutoSize`, `WordWrap`, text alignment, `Align`, `Anchors`, tab order, and grid column order — that is, exactly the set of properties the pack is allowed to change.

14.3 What is never restored

Color. The applicator does not set colors, so restoring one can only undo something the application itself did. An application that paints its own colors after a form is shown would have them overwritten with design-time values on every language change. The rule is general and worth stating as such: *what we never changed, we never restore*.

14.4 The language manager and form discovery

The component in the customer's application is a language manager: it loads packs from a folder, exposes the available languages, applies a chosen one, and remembers the choice.

`DAT.Runtime.Preference` stores that choice per user.

Applying a language to the forms that are already open is the easy half. The harder half is a form created *after* the language was chosen — a dialog opened from a button — which must be translated when it is shown. The two frameworks solve this by entirely different mechanisms:

	VCL	FireMonkey
Finding open forms	<code>Screen.Forms</code>	<code>Screen.Forms</code> and <code>Screen.PopupForms</code>

Noticing a new form	Hooks the window procedure	Subscribes to TFormBeforeShownMessage through TMessageManager
Noticing a closed form	Window destruction	TFormReleasedMessage

Because those mechanisms share nothing, one of them working says nothing at all about the other. Each has its own test (section 15.6).

15. Contracts: how this product is kept honest

This section describes the practice that most distinguishes this codebase. It is worth reading even by someone who intends to change nothing, because the rules here explain why the tests are shaped the way they are.

15.1 What a contract is

A layout contract is three files: a small purpose-built form, a catalog naming the translated text for it, and an expectation file stating in numbers what the planner must do with them. Assertions are numeric because layout is numeric — a right edge that must not move, a control that must keep its place, a caption that must still hold its text.

The forms are purpose-built rather than taken from a real application, so each rule is proved on the *shape* of a problem rather than on one project's particular arrangement of controls.

64 layout contracts currently run, alongside 3 form-scan fixtures, 9 pascal-scan checks, and **30 test harnesses**.

15.2 A contract must fail first

The single most important rule. A contract whose expected values are copied from what the code currently does proves nothing; it records behavior instead of requiring it.

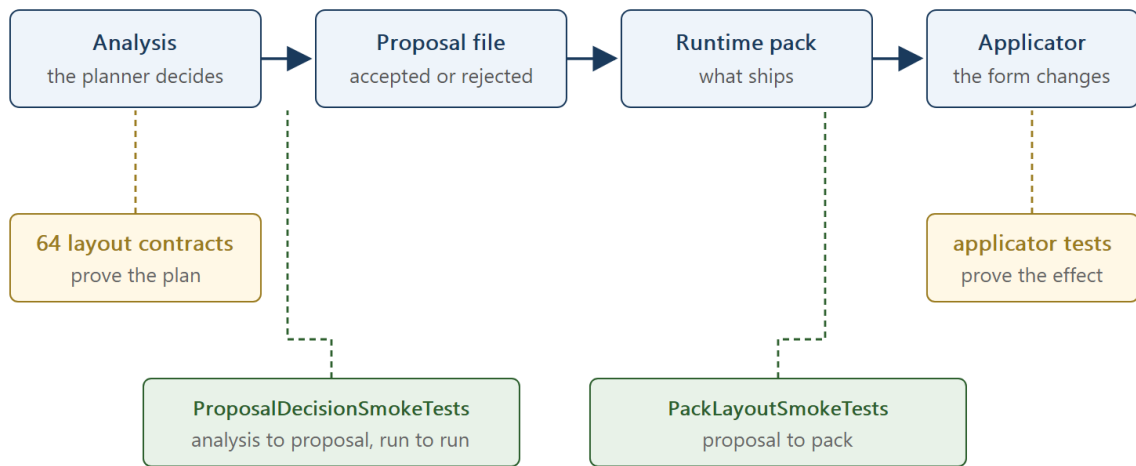
Two contracts in this project had to be rebuilt for exactly that fault. Both passed from the day they were written, and both were worthless: one flipped a constant on a fixture that never reached that code path, and the other used a caption that lacked the structure the rule depended on.

The discipline is therefore: write the assertion, watch it fail for the right reason, then make it pass. Where a test is written after the code, which does happen, it is verified by deliberately disabling the code and confirming that the test fails. A test that has never been seen to fail is an untested test.

15.3 Contracts must cross seams, not sit at the ends

Figure 2 — Where the tests sit, and the seams between them

A value can be correct at both ends of the pipeline and absent in the middle.



Tests at the ends pass while the middle is empty. Only a test that crosses a seam can see the hole.

The hardest defects this product has had all shared one shape: **a value correct at both ends of the pipeline and absent in the middle.**

Three separate failures removed the right-to-left feature before anyone noticed:

1. One applicator held its own copy of the allowed-property list and dropped every mirroring rule.
2. The pack exporter held another copy and dropped them again.
3. The auto-accept rule held a third, so the proposals were created *pending*, and the saved-decision restore made that permanent.

Every one of those was invisible to every test that existed, because the tests sat at the ends. The layout contracts passed because the plan was right. The applicator tests passed because they were handed a pack written by hand. Nothing crossed the joins between them.

Three tests now do. PackLayoutSmokeTests goes proposal to pack, the contract harness can assert a proposal's *decision* as well as its value, and ProposalDecisionSmokeTests goes run to run. The first of those failed on seven of eight checks the day it was written, which is what a seam test is supposed to do.

15.4 A test must not disturb what it measures

A diagnostic added to the VCL runtime read TMenu.Handle in order to report the native right-to-left flag. TMenu.Handle creates and populates the menu if it does not already exist, and doing that in the middle of applying a language **destroyed the translated menu captions**. The runtime smoke test failed within seconds of the diagnostic being switched on, which is exactly what that test is for. The diagnostic was changed to ask Windows for the menu already attached to the window, which creates nothing.

15.5 Guards around the suite

- **Stale-harness rejection.** If any analyzer source is newer than the contract executable, the run stops rather than reporting on code that is no longer the code being changed.
- **Coverage enforcement.** A fixture with no expectation file fails the run, so a rule cannot quietly exist without a contract.
- **Source encoding.** `tools\check_source_encoding.ps1` fails the build if any `.pas`, `.dpr`, or `.dpk` holds a character outside ASCII without a byte order mark, because Delphi reads such a file in the ANSI codepage and its text arrives wrong. It has caught real defects that would otherwise have shipped.
- **Rebuild after every change.** All four configurations, every time.

15.6 Framework parity

The 64 contracts cover **33 distinct behaviors**: 31 proven on both frameworks, 2 on the VCL only, and none on FireMonkey only. The two are VCL-only by design rather than by omission, and Appendix B says why for each.

Parity is not the same as identity, and this is the trap to understand before writing a twin. Because the planner measures through a seam, **the same rule legitimately produces different numbers on each framework**. A fixture built by copying its twin's font size reproduces that twin's numbers but not its *situation*: text that needs a font reduction under one engine may fit comfortably under the other, so the rule is never asked to do anything and the contract watches nothing. Twins must be reasoned from what the layout should be, and each must be seen to fail before the code is right.

The test harnesses pair the same way: seven matched pairs, one VCL-only harness covering MDI, which is a VCL concept with no FireMonkey equivalent, and the rest framework-neutral.

16. Extending the product

This section is procedural. It assumes the preceding sections have been read.

16.1 Adding a layout rule

Write the contract first. Create the three fixture files under `contracts\layout`, run the suite, and confirm that the new contract fails for the reason you expect. Then add the rule to the settling pass in `DAT.Review.Localization`, making it trial-and-revert like its neighbors. Run the whole suite, not only the new contract: a settling rule that fights an existing one shows up as a failure elsewhere, and that failure is information.

If the rule applies to both frameworks, write the twin at the same time, and read section 15.6 before choosing its numbers.

16.2 Adding a runtime layout property

This is the change most likely to be silently lost, because it crosses three boundaries. In order:

1. Add the property to `IsRuntimeLayoutProperty` in `DAT.Runtime.LanguagePack`. This is the only list; do not add a second one anywhere.
2. Emit a proposal for it in the planner, and make sure it is created accepted if it should apply by default.
3. Apply it in **both** applicators, in the right place in the fixed order of section 14.1.
4. Add it to the snapshot, or a later language change will not be able to undo it.
5. Delete any saved proposal file from earlier runs while testing, or a stale *pending* entry will veto the new property and the feature will appear not to work (section 13.1).

Then extend `PackLayoutSmokeTests`, which is the test that crosses from proposal to pack and is the one that would have caught each of the historical failures in section 15.3.

16.3 Adding a translation provider

Implement the transport in a new unit beside `DAT.Provider.Client` and reuse the four units around it: placeholders, batching, language codes, and retry are not provider-specific and should not be reimplemented. The provider-specific work is the request format, the response format, the error body, and the list of language codes the service accepts. Extend `DAT.Provider.LanguageCodes` with that list rather than passing codes through, and read section 9.2 for why dropping a region is the safe direction.

16.4 Adding a language

A language needs an entry in the Wizard's list with its text direction, and, if it is a compounding

language, a hyphenation dictionary under the shared store (section 10.1). Everything else follows from the catalog: the direction drives mirroring, and the locale block drives format settings.

If the language is right-to-left, expect to verify mirroring on a machine whose Windows locale supports those languages, for the reason given in section 12.4.

16.5 Adding a test harness

Harnesses live in `tools\tests` as console programs that compile directly against the product's source and exit non-zero on failure. Keep the framework-specific ones in pairs. Where the two frameworks genuinely behave differently, assert the difference rather than papering over it: a test that asserted identical soft-hyphen behavior on both frameworks (section 10.3) would be asserting that one of the two runtimes has a defect.

17. Build and release validation

All four configurations — Win32 and Win64, Debug and Release — are rebuilt after every change. The release validation script requires one uninterrupted pass of the complete matrix, plus the Studio launch and self-localization smoke tests.

The full suite comprises the 64 layout contracts, 3 form-scan fixtures, 9 pascal-scan checks, and the 30 harnesses covering the foundation, scanner, catalog, runtime, validation, and export paths; the VCL and FireMonkey runtime smoke tests; the four language-manager suites; and the focused harnesses for retry, language codes, context batching, proposal decisions, pack export, placeholders, hyphenation, context, wrap, discovery, and right-to-left on both frameworks.

Runtime **packages** must be rebuilt separately when a runtime unit changes. An application that links them will not otherwise pick up the change, and the symptom is a fix that appears not to work.

18. Known limits

Recorded in full in `docs\guides\Fix List.md`. The ones an engineer should know before planning work:

- **No translation memory** and no TMX or TBX interchange. This is the largest functional gap against commercial alternatives and the main obstacle to a customer migrating existing translation assets in.
- **Runtime-composed strings** cannot be translated without a call in the target application, and the product does not yet report which strings those are, even though it holds the file and line for each.
- **Rows at a fixed pitch cannot grow.** Where a language needs more width per member than the designed pitch allows, the last members are wrapped or cramped rather than the row being re-

pitched.

- **First run shows the source language** whatever the machine is set to; nothing consults the operating system's UI language.
- **The settling pass can disturb an image-backed form.** Widening a control or growing a font is safe on an ordinary form and not on one where text sits over artwork. Such a form usually wants its words translated and its geometry frozen.
- **CJK is untested.** Chinese, Japanese, and Korean do not wrap on spaces and do not hyphenate, and they usually run shorter than English rather than longer, so the settling pass would be exercised in a direction it has never seen.
- **Validation breadth.** Nearly every significant defect this product has found came from running it against a real application rather than from reasoning. The observed rate suggests that each additional real application will find another batch.

Appendix A — Unit inventory

Unit	Purpose
core	
DAT.Core.Types	Catalog, entry, locale, and enumeration types
DAT.Core.CatalogJson	Catalog read and write
DAT.Core.RuntimePack	Builds the shipped pack
DAT.Core.Glossary	Per-project approved terms
DAT.Core.SharedDictionary	Per-language wording shared across applications
DAT.Core.Hyphenation	Per-language break dictionaries
DAT.Core.Terminology	Authoritative term resolution
DAT.Core.TranslationWorkspace	Where every artifact lives
DAT.Core.ProjectDetection	Recognizing a Delphi project and its framework
DAT.Core.AITranslation	The copy-and-paste AI workflow
scan	
DAT.Scan.Project	Drives the scan
DAT.Scan.FormText	Reads .dfm and .fmx
DAT.Scan.PascalResources	Reads .pas
DAT.Scan.Context	Writes each string's context sentence
DAT.Scan.DomainProfile	Vocabulary and word-sense resolution
DAT.Scan.Rules	Text ownership classification
DAT.Scan.Quality	Quality checks on scanned text
DAT.Scan.TextCodec	Source file encoding
DAT.Scan.CatalogMerge	Merging a re-scan into an edited catalog
DAT.Scan.Types	Scan-layer types
review	
DAT.Review.Localization	The layout planner
DAT.Review.TextMeasurement	The measurement seam
DAT.Review.TextMeasurement.GDI	VCL measurement
DAT.Review.TextMeasurement.FMX	FireMonkey measurement
DAT.Review.CodeGeometry	Controls the application positions itself

provider	
DAT.Provider.Client	Service transport
DAT.Provider.Placeholders	Format specifier protection
DAT.Provider.Batching	Grouping by shared context
DAT.Provider.LanguageCodes	Codes a service will accept
DAT.Provider.Retry	Rate limits and backoff
DAT.Provider.CredentialStore	API keys
DAT.Provider.Settings	Provider settings
DAT.Provider.Types	Provider-layer types
runtime (ships in the customer's application)	
DAT.Runtime.LanguagePack	Pack loading; the one allowed-property list
DAT.Runtime.VCL	Applying a pack to a VCL form
DAT.Runtime.FMX	Applying a pack to a FireMonkey form
DAT.Runtime.Manager	Language selection and format settings
DAT.Runtime.Preference	Remembering the chosen language
components (ship in the customer's application)	
DAT.Components.Core	Shared language-manager behavior
DAT.Components.VCL	VCL adapter
DAT.Components.FMX	FireMonkey adapter
DAT.Components.VCL.LanguageSelector	Optional bound selector, VCL
DAT.Components.FMX.LanguageSelector	Optional bound selector, FireMonkey
validation, integration, studio	
DAT.Validation.Catalog	The export gate
DAT.Integration.*	Component kits, packages, source integration, deployment
DAT.Studio.*	Main form, Setup Wizard, translation, localization review

Appendix B — Contract inventory and framework parity

Contract names are file identifiers and are reproduced exactly as they appear on disk.

B.1 Proven on both frameworks — 31 behaviors

a_button_gets_room_for_its_caption; a_caption_stops_at_the_button_beside_it;
a_heading_widens_before_it_wraps; a_long_word_gets_room; a_paragraph_stays_on_the_form;
button_above_grid_padding; button_keeps_its_place_when_text_grows; button_row_keeps_even_pitch;
caption_above_field_takes_its_column; caption_far_wider_than_its_box;
centred_heading_widens_about_centre; checkbox_caption_inside_container; checkbox_caption_note;
code_positioned_control_is_left_alone; container_keeps_its_size; designed_overlap_is_left_alone;
email_label_button_padding; frame_grows_and_says_so; grid_headers_fit;
intro_paragraph_wraps_compact; left_caption_takes_left_margin; long_button_and_field_stack;
media_button_row_container; memo_label_pair; preserve_label_font;
right_aligned_caption_grows_leftward; right_to_left_flips_alignment_and_anchors;
right_to_left_mirrors_the_form; stacked_paragraphs_share_a_size; transport_buttons_keep_their_order;
wrapped_text_is_balanced.

B.2 VCL only, by design — 2 behaviors

Contract	Why there is no twin
inherited_font_is_the_forms_font	Font inheritance down the object tree was changed for the VCL alone, on VCL evidence. FireMonkey inherits through StyledSettings and TextSettings instead, so its equivalent is a different contract rather than a twin.
a_grid_does_not_end_the_form	Guards a .dfm parsing hazard specifically: a collection written as item ... end blocks, whose end lines emptied the object stack. FireMonkey files have no such syntax.

B.3 Harness parity

Thirty harnesses: seven matched framework pairs — design streaming, language manager, manager lifecycle, right-to-left, runtime smoke, discovery, and wrap — one VCL-only harness covering MDI, which has no FireMonkey equivalent, and the remainder framework-neutral, covering scanning, providers, packs, hyphenation, context, and the contract runners themselves.

Appendix C — Files, folders, and where things live

What	Where
Development catalog	%LOCALAPPDATA%\DelphiAppTranslationStudio\Workspaces\ <project>\development< td=""></project>\development<>
Runtime packs	...\Workspaces\ <project>\languages, beside="" deployed="" executable<="" localization\languages="" td="" the="" to=""></project>\languages,>
Layout proposal and review	export\localization-review\ <project>\<language>< td=""></project>\<language><>
Shared dictionaries, terms, hyphenation	C:\Users\Public\Documents\Delphi App Translation
Language preference (target application)	%LOCALAPPDATA%
Layout contracts	contracts\layout
Test harnesses	tools\tests
Engineering notes and Fix List	docs\guides

This guide describes the product as it stood on August 21, 2026. Where it disagrees with the code, the code is right and this document is stale. The Engineering Notes in docs\guides\Engineering Notes.md carry the running record of change.